

Playwright + TypeScript Framework

Interview guide: architecture, design choices, and likely questions

1. 30-second pitch

"I built a UI automation framework with **Playwright Test and TypeScript**, using the **Page Object Model**. Locators and page actions live in page classes under `pages/`. Test data lives in the `data/` layer, typed with TypeScript interfaces, and I use **Faker** to generate unique data such as new customer accounts. Environment settings like the base URL and browsers are kept in `playwright.config.ts`, and secrets are loaded through **dotenv**. The same tests run in parallel on **Chromium, Firefox and WebKit** and produce an **HTML report**."

2. Folder structure

<code>playwright.config.ts</code>	→ config (base URL, browsers, parallel runs, retries, reporter)
<code>.env + dotenv</code>	→ secrets / environment values
<code>pages/</code>	→ Page Objects (LoginPage, MyAccountPage)
<code>data/</code>	→ test data + TS interfaces (User, customerInfo) + Faker
<code>tests/</code>	→ specs: steps + assertions only

3. The layers

3.1 Config layer: `playwright.config.ts`

- `baseUrl` is set here, so tests call `page.goto('/')`. To move to QA, UAT or prod, you change one line or an env variable, and no test changes.
- `fullyParallel: true` means tests run in parallel across workers. `retries: process.env.CI ? 2 : 0` means retries only happen on CI. `trace: 'on-first-retry'` saves a trace of the failed run for debugging.
- `projects` runs the same suite on 3 browser engines.
- **Selenium bridge**: this file does the job of `testng.xml`, `config.properties`, and your DriverFactory together.

3.2 Page Object layer: LoginPage.ts, MyAccountPage.ts

- Locators are created in the constructor as `Locator` objects. A `Locator` is **lazy**: it doesn't search the page when it's created, only when you act on it. So there's nothing like a `StaleElementReferenceException`, and you don't need `PageFactory`.
- `doLoginWith()` returns `Promise<MyAccountPage>`. This is **fluent page navigation**: the method that moves you to the next page hands back that page's object.
- Locators mostly use **role-based** strategies (`getByRole`), which follow how users and accessibility tools see the page and break less often than `XPath`.

```
export class LoginPage {
  page: Page
  emailTextBoxLocator: Locator
  // ...
  constructor(page: Page) {
    this.page = page;
    this.emailTextBoxLocator = page.locator('#email');
  }
  async doLoginWith(email: string, password: string): Promise<MyAccountPage> {
    // click Sign in, fill email + password, submit
    return new MyAccountPage(this.page);
  }
}
```

3.3 Test data layer: data/

- `user.type.ts` / `customerInfo.type.ts` are **interfaces** that act as contracts for the shape of the data. If a field is missing or has the wrong type, you get a compile-time error.
- `createFakeCustomerData()` uses `Faker`, so every sign-up run uses a new email and doesn't fail on "email already exists".
- **Selenium bridge**: this is like POJOs plus a `@DataProvider`, but with type safety checked at compile time.

3.4 Test layer: tests/login3.spec.ts

```
test('login test', async ({ page }) => {
  const user = getUserData();
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  const myAccountPage = await loginPage.doLoginWith(user.emailAddress, user.password);
  await expect(myAccountPage.userAccountLocator).toBeVisible();
});
```

- The test only reads as steps plus an assertion. Selectors don't appear in the test.
- `expect(...).toBeVisible()` is a **web-first assertion**: it retries until the element appears or the timeout runs out. That's why the framework has no `Thread.sleep` and no `WebDriverWait`.

4. Questions they'll likely ask

Question	Your answer
Why Playwright over Selenium?	It waits for elements automatically, its assertions retry, it runs tests in parallel and in isolated browser contexts out of the box, it has the trace viewer, and it can do API testing in the same tool. It also talks to the browser directly over the DevTools protocol instead of through WebDriver's HTTP layer.
Where does the <code>page</code> object come from?	It's a fixture , a bit like <code>@BeforeMethod</code> . Playwright gives each test a fresh <code>BrowserContext</code> and <code>page</code> and cleans them up afterwards.
Why <code>await</code> everywhere?	Every browser action is async and returns a Promise. If you forget one, the next step can run before the previous one finishes, which makes tests flaky.
Assertions in page objects or tests?	In tests. Page objects expose locators and state, and the test decides what counts as pass or fail.
How do you switch environments?	Through <code>baseURL</code> in the config, driven by environment variables loaded with <code>dotenv</code> . Tests never contain URLs.
How do you debug a failure?	Open the HTML report, then the trace viewer (<code>npx playwright show-trace</code>), which shows DOM snapshots, network calls and console output for each step. Locally, use <code>--debug</code> or <code>--ui</code> mode.

5. Planned improvements (next steps)

An interviewer who looks closely will spot these, so be ready to present them as your roadmap:

1. **Read credentials from `.env`** (`process.env.EMAIL!`) instead of hardcoding them in `user.data.ts`.
2. **Custom fixtures** (`test.extend<{ loginPage: LoginPage }>`), so a test can just receive (`{ loginPage }`) instead of creating page objects by hand. This is the question most likely to come up for a senior role.
3. **Log in once and reuse the session**: log in in a setup project, save `storageState`, and skip the UI login in every test.
4. **Move the remaining tests onto page objects** (e.g. `createAccount.spec.ts`).
5. **Remove user-specific locators**, like the account-name link tied to one user's data.
6. **Add CI** with a GitHub Actions workflow that runs the suite and publishes the HTML report.

Once 1 and 2 are done, you can describe the framework as **"POM + custom fixtures + env-driven config"**, which is what interviewers expect at the SDET level.